

Voice Agent Service & Clio CRM (Grow & Manage) Integration

Complete Technical Architecture, Dual-Pattern Integration Blueprint, API Schemas & Production Deployment

Author: AI Voice Platform Architecture Team

Target APIs: Clio Grow Public Inbox API & Clio Manage REST API v4

1. Architectural Overview & Integration Strategies

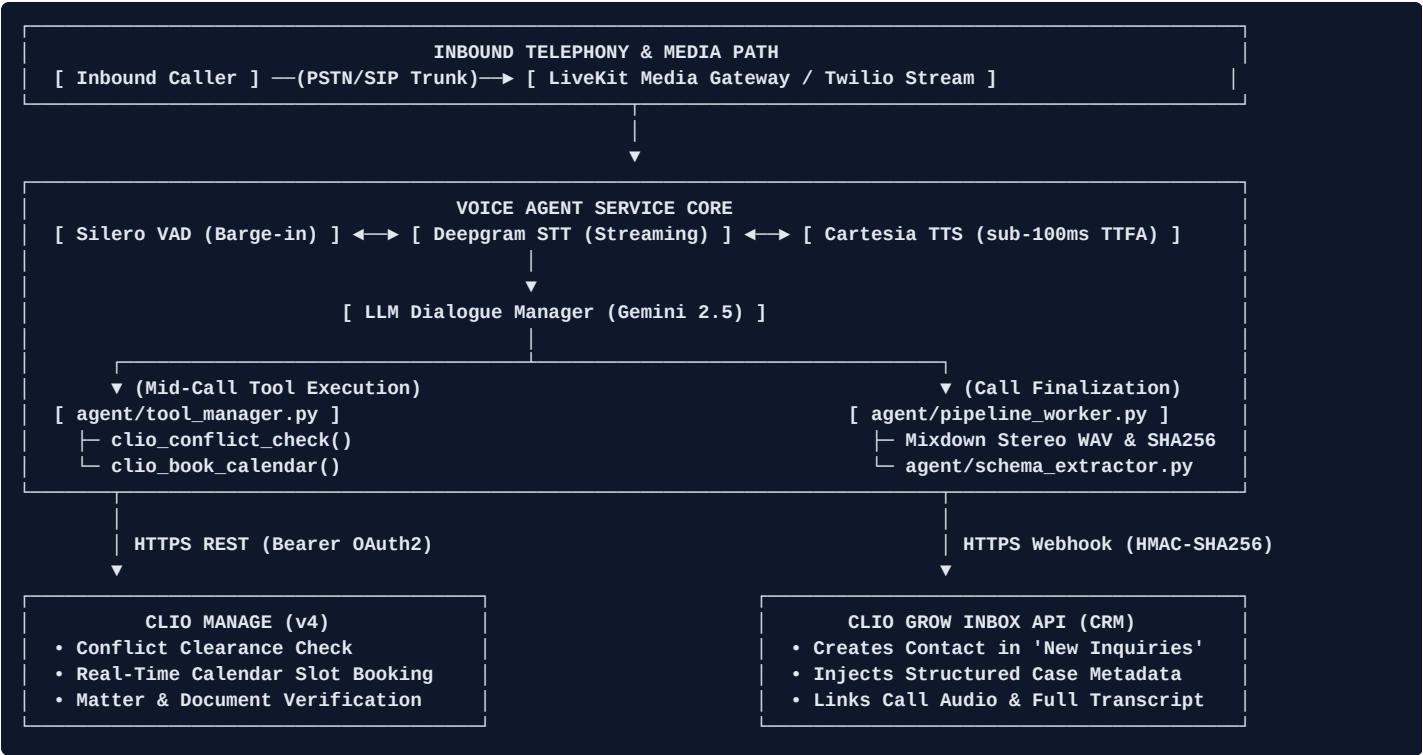
Integrating the **Voice Agent Service** with **Clio** bridges real-time conversational telephony with the firm's legal practice management system. This eliminates manual intake entry, accelerates lead response times to sub-second speeds, and guarantees data integrity under legal compliance standards.

The architecture supports two complementary integration patterns:

- Pattern A: Post-Call Asynchronous Ingestion (Clio Grow CRM):** When a call finishes, the Voice Agent pipeline extracts structured intake fields using JSON Schema, creates a compliant lead record via Clio Grow's Inbox API, and attaches the call recording URL and conversation transcript.
- Pattern B: Mid-Call Synchronous Tool Execution (Clio Manage API v4):** While the caller is actively speaking, the LLM dialogue manager invokes registered function tools to execute live conflict checks and query real-time attorney calendar availability to book consultations.

Recommended Standard: Deploy **Pattern A** for 100% of inbound calls to guarantee lead capture, and layer **Pattern B** for advanced intake where the agent must check attorney calendar slots and schedule consultations in real time.

2. End-to-End System Topology



3. Pattern A: Post-Call Ingestion Pipeline (Clio Grow API)

This pattern leverages the Voice Agent's built-in `agent/schema_extractor.py` and `agent/webhook_dispatcher.py` to deliver clean, validated lead records to Clio Grow.

1. Data Extraction via LEGAL_INTAKE_SCHEMA

Upon call teardown, the pipeline worker runs the transcript through the legal schema extractor, resolving typed business entities:

```
# Built-in agent/schema_extractor.py LEGAL_INTAKE_SCHEMA outputs:
{
  "client_name": "Eleanor Vance",
  "contact_phone": "+15550192834",
  "contact_email": "eleanor.vance@example.com",
  "case_type": "personal_injury",
  "injury_date": "2026-09-15",
  "needs_urgent_consultation": true,
  "retainer_discussed": false
}
```

2. Webhook Event Dispatch (HMAC-SHA256 Signed)

The webhook dispatcher fires a `call.extracted` event to the integration middleware with replay prevention headers: `X-Signature-256`, `X-Webhook-Timestamp`, and `X-Webhook-Delivery`.

3. Clio Grow Ingestion API Payload

The middleware transforms the extracted event into Clio Grow's standard `/api/v1/inbox_leads` format:

```
POST https://grow.clio.com/api/v1/inbox_leads
Authorization: Bearer <CLIO_GROW_INBOX_TOKEN>
Content-Type: application/json

{
  "inbox_lead": {
    "first_name": "Eleanor",
    "last_name": "Vance",
    "phone": "+1-555-019-2834",
    "email": "eleanor.vance@example.com",
    "message": "Motor vehicle collision on 2026-09-15. Severe cervical strain. Opposing vehicle: Commercial delivery truck",
    "source": "AI Voice Agent - Inbound Telephony",
    "referrer": "Automated Phone Intake",
    "custom_field_values": [
      {"custom_field_id": 48201, "value": "personal_injury"},
      {"custom_field_id": 48202, "value": "2026-09-15"},
      {"custom_field_id": 48203, "value": "https://storage.firm.com/recordings/call_20260923_rec.wav"}
    ]
  }
}
```

4. Pattern B: Mid-Call Tool Calling (Clio Manage API v4)

When an intake call requires immediate verification or booking, the Voice Agent's `agent/tool_manager.py` executes mid-call tools within a sub-200ms round-trip window while playing subtle audio fillers to prevent dead air.

1. Tool Definition: Clio Conflict Check

```
# Tool schema registered in agent/tool_manager.py
CLIO_CONFLICT_TOOL = {
    "type": "function",
    "function": {
        "name": "check_clio_conflict",
        "description": "Checks Clio Manage database for legal conflicts of interest against adverse party or person name."
        "parameters": {
            "type": "object",
            "properties": {
                "party_name": {"type": "string", "description": "Full name of opposing party, company, or driver."}
            },
            "required": ["party_name"]
        }
    }
}
```

2. Tool Implementation: Clio API v4 Endpoint

```
import aiohttp
import os

CLIO_MANAGE_BASE = "https://app.clio.com/api/v4"

async def check_clio_conflict(party_name: str) -> dict:
    """Queries Clio Manage v4 /conflicts endpoint to detect ethical clashes."""
    token = os.getenv("CLIO_MANAGE_ACCESS_TOKEN")
    headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}
    params = {"query": party_name}

    async with aiohttp.ClientSession() as session:
        async with session.get(f"{CLIO_MANAGE_BASE}/conflicts.json", headers=headers, params=params) as resp:
            if resp.status == 200:
                data = await resp.json()
                matches = data.get("data", [])
                has_conflict = len(matches) > 0
                return {
                    "conflict_detected": has_conflict,
                    "match_count": len(matches),
                    "adverse_party": party_name
                }
            return {"conflict_detected": False, "error": f"HTTP {resp.status}"}
```

3. Tool Definition: Clio Consultation Scheduler

```
async def book_clio_consultation(client_name: str, phone: str, start_iso: str, attorney_id: int) -> dict:
    """Creates a calendar entry in Clio Manage for an initial attorney consultation."""
    token = os.getenv("CLIO_MANAGE_ACCESS_TOKEN")
    headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}
    payload = {
        "data": {
            "summary": f"Initial Voice Intake Consultation: {client_name}",
            "description": f"Scheduled automatically by AI Voice Agent. Caller contact: {phone}",
            "start_at": start_iso,
            "end_at": add_minutes_iso(start_iso, 30),
            "calendar_owner": {"id": attorney_id}
        }
    }

    async with aiohttp.ClientSession() as session:
        async with session.post(f"{CLIO_MANAGE_BASE}/calendar_entries.json", headers=headers, json=payload) as resp:
            res_json = await resp.json()
            return {"success": resp.status == 201, "booking_id": res_json.get("data", {}).get("id")}
```

5. Ready-to-Deploy Python Integration Connector

Below is the complete standalone production middleware script (FastAPI) that consumes Voice Agent webhooks, verifies HMAC signatures, and syncs directly into Clio Grow:

```
from fastapi import FastAPI, Request, Header, HTTPException, BackgroundTasks
import hmac, hashlib, os, httpx, logging

app = FastAPI(title="Voice Agent Clio Connector")
WEBHOOK_SECRET = os.getenv("VOICE_AGENT_WEBHOOK_SECRET", "dev_secret_key").encode()
CLIO_GROW_TOKEN = os.getenv("CLIO_GROW_INBOX_TOKEN")

async def forward_to_clio_grow(lead_data: dict, transcript: str, audio_url: str):
    clio_payload = {
        "inbox_lead": {
            "first_name": lead_data.get("client_name", "Unknown").split()[0],
            "last_name": " ".join(lead_data.get("client_name", "Caller").split()[1:]) or "Lead",
            "phone": lead_data.get("contact_phone", ""),
            "email": lead_data.get("contact_email", ""),
            "message": f"Case Type: {lead_data.get('case_type')}\\nDate: {lead_data.get('injury_date')}\\n\\nTranscript:\\n{tr",
            "source": "Voice Agent Intake",
            "referrer": audio_url
        }
    }
    async with httpx.AsyncClient() as client:
        r = await client.post("https://grow.clio.com/api/v1/inbox_leads",
                              headers={"Authorization": f"Bearer {CLIO_GROW_TOKEN}"},
                              json=clio_payload, timeout=10.0)
        logging.info("Clio Grow Sync Response: %s %s", r.status_code, r.text)

@app.post("/webhooks/voice-agent")
async def handle_call_webhook(req: Request, bg: BackgroundTasks, x_signature_256: str = Header(None), x_webhook_timestamp: str = Header(None)):
    body = await req.body()
    # 1. Verify HMAC-SHA256 signature
    expected = hmac.new(WEBHOOK_SECRET, f"{x_webhook_timestamp}".encode() + body, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(f"sha256={expected}", x_signature_256 or ""):
        raise HTTPException(status_code=401, detail="Invalid signature")

    event = await req.json()
    if event.get("event") in ("call.completed", "call.extracted"):
        data = event.get("data", {})
        bg.add_task(forward_to_clio_grow, data.get("extracted_fields", {}), data.get("transcript", ""), data.get("recording_url", ""))
    return {"status": "accepted"}
```

6. Production Deployment & Verification Checklist

Deployment Step	Configuration / Action	Verification Acceptance Criteria
1. Obtain Clio Grow Token	Navigate to <i>Clio Grow Settings > Integrations > Public API > Inbox Lead Token</i> . Copy Bearer secret.	Token responds with <code>HTTP 200/201</code> on test POST to <code>grow.clio.com/api/v1/inbox_leads</code> .
2. Configure OAuth for Manage	Register an Application in <i>Clio Manage Settings > Apps > Developer</i> . Set redirect URI and scopes (<code>read_conflicts</code> , <code>write_calendar</code>).	OAuth token refresh cycle passes automated check every 60 minutes.
3. Register Webhook Endpoint	Register the connector URL in <code>agent/webhook_dispatcher.py</code> via <code>registry.register_endpoint()</code> with unique secret.	Run <code>python3 scripts/verify_webhooks.py</code> ; confirms 5/5 delivery attempts pass signature test.
4. Calibrate Schema Extractor	Ensure <code>LEGAL_INTAKE_SCHEMA</code> in <code>agent/schema_extractor.py</code> contains firm-specific practice custom fields.	Run <code>python3 scripts/verify_extraction.py</code> ; ensures >90% confidence on name, phone, and case facts.
5. End-to-End Live Call Test	Place inbound call via WebRTC dialpad (<code>http://localhost:8091/softphone.html</code>) or Twilio SIP trunk. Complete test intake.	New Lead Card instantly appears in Clio Grow Inbox with extracted phone, email, summary, and audio link.

7. Security, Compliance & Error Handling

- **HIPAA & Attorney-Client Privilege:** Inbound call recordings can be encrypted at rest using AES-256 before generating presigned storage URLs. Sensitive PCI data (e.g. credit card numbers) is automatically silenced via the Voice Agent's 1400 Hz tone compliance pause engine.
- **Dead-Letter Queue (DLQ) & Retries:** If Clio's API experiences downtime or 429 rate limiting, the Voice Agent's webhook dispatcher utilizes exponential backoff retries with jitter (1s, 2s, 4s, 8s, up to 30s) and parks unacknowledged events in disk-persisted state.
- **Replay Attack Prevention:** Webhook requests verify signed timestamps within a 300-second window, strictly rejecting expired or replayed delivery IDs.